



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 04/10

MySQL + SQLAlchemy

Bases relationnelles + ORM Python async

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - C'est quoi une BDD relationnelle ?	3
Chapitre 2 - SQL : les bases	6
Chapitre 3 - Cles primaires et etrangeres	10
Chapitre 4 - Normalisation (1NF, 2NF, 3NF)	13
Chapitre 5 - Les 16 tables de fsbm_db	16
Chapitre 6 - Relations entre tables	22
Chapitre 7 - Index et performance	25
Chapitre 8 - C'est quoi un ORM ?	28
Chapitre 9 - SQLAlchemy 2.0	31
Chapitre 10 - SQLAlchemy async (aiomysql)	35
Chapitre 11 - CRUD avec SQLAlchemy	38
Chapitre 12 - Pagination	42
Chapitre 13 - Conclusion	45

Chapitre 1 - C'est quoi une BDD relationnelle ?

Une **base de donnees relationnelle** (RDBMS) stocke les donnees dans des **tableaux** (tables) lies entre eux par des **cles**. Le concept date des annees 70 (Edgar F. Codd, IBM).

■ **ANALOGIE** : Une BDD relationnelle est comme un **classeur Excel** avec plusieurs feuilles : une feuille 'Etudiants', une feuille 'Filières', une feuille 'Notes'. Les feuilles sont **liees** : chaque etudiant a une filiere, chaque note appartient a un etudiant + un module.

1.1 Les 4 SGBD relationnels populaires

SGBD	Createur	Forces
MySQL	Oracle (rachat de Sun)	Tres rapide, gratuit, populaire
PostgreSQL	Communaute	Tres complet, SQL standard, JSONB
SQL Server	Microsoft	Excellent pour env Microsoft
Oracle DB	Oracle	Entreprise, tres cher
SQLite	D. Richard Hipp	Embarque (1 fichier)

■ Le cahier des charges PFE imposait **MySQL**. C'est aussi le SGBD le plus utilise au Maroc (WordPress, banques, e-commerce).

1.2 Pourquoi MySQL pour notre projet ?

- **Impose par le PFE** - on respecte le cahier des charges
- **Gratuit et open source**
- **Performant** pour les volumes moyens (millions de rows)
- **Compatible** avec phpMyAdmin, MySQL Workbench, DBeaver
- **Largement supporte** par tous les hebergeurs
- **Ecosysteme** Python tres mur (PyMySQL, aiomysql, SQLAlchemy)

Chapitre 2 - SQL : les bases

2.1 C'est quoi SQL ?

SQL = Structured Query Language. C'est le langage standardise (ISO) pour parler aux BDD relationnelles. Tous les SGBD relationnels le supportent (avec des variantes).

2.2 Les 4 categories d'instructions

Categorie	Acronyme	Exemples
Definition	DDL	CREATE, ALTER, DROP
Manipulation	DML	SELECT, INSERT, UPDATE, DELETE
Controle	DCL	GRANT, REVOKE
Transaction	TCL	BEGIN, COMMIT, ROLLBACK

2.3 CREATE TABLE

```
CREATE TABLE departments (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  code       VARCHAR(20) NOT NULL UNIQUE,
  name       VARCHAR(150) NOT NULL,
  description TEXT,
  head_name  VARCHAR(150),
  head_email VARCHAR(150),
  color_hex  VARCHAR(7) DEFAULT '#1C3F6E',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

2.4 INSERT

```
INSERT INTO departments (code, name, head_name) VALUES
('MI', 'Departement de Mathematiques et Informatique', 'Pr. Mohammed ALAOUI'),
('PH', 'Departement de Physique', 'Pr. Khadija BENNANI'),
('CH', 'Departement de Chimie', 'Pr. Youssef LAHLOU'),
('BIO', 'Departement de Biologie', 'Pr. Fatima ZAHIDI'),
('GEO', 'Departement de Geologie', 'Pr. Hassan BERRADA');
```

2.5 SELECT (lecture)

```
-- Tout :
SELECT * FROM departments;

-- Colonnes specifiques :
SELECT code, name FROM departments;

-- Avec filtre :
SELECT * FROM departments WHERE code = 'MI';
```

```
-- Avec tri :
SELECT * FROM departments ORDER BY name ASC;

-- Avec limite :
SELECT * FROM departments LIMIT 10 OFFSET 20; -- page 3 de 10

-- Avec count :
SELECT COUNT(*) FROM students WHERE statut = 'ACTIF';

-- Avec group by :
SELECT filiere_id, COUNT(*) AS nb_etudiants
FROM students
GROUP BY filiere_id
ORDER BY nb_etudiants DESC;
```

2.6 UPDATE et DELETE

```
UPDATE departments
SET head_phone = '+212 522 70 46 71'
WHERE code = 'MI';

DELETE FROM faq_items WHERE consultations = 0;
```

[X] TOUJOURS mettre une clause WHERE sur UPDATE et DELETE. Sans, tu modifies/supprimes TOUTE la table !

2.7 JOIN

Les JOINS permettent de combiner plusieurs tables :

```
-- INNER JOIN : intersection (etudiants qui ont une filiere)
SELECT s.cne, s.first_name, s.last_name, f.code AS filiere
FROM students s
INNER JOIN filieres f ON s.filiere_id = f.id
WHERE f.code = 'SMI';

-- LEFT JOIN : tous les etudiants, meme sans filiere
SELECT s.first_name, f.code AS filiere
FROM students s
LEFT JOIN filieres f ON s.filiere_id = f.id;
```

Chapitre 3 - Cles primaires et etrangeres

3.1 Cle primaire (PRIMARY KEY)

La cle primaire est l'**identifiant unique** d'une ligne. Une seule par table.

- **Unique** : aucune ligne n'a la meme valeur
- **Non null** : toujours definie
- **Indexee** automatiquement (recherche tres rapide)
- Souvent un BIGINT AUTO_INCREMENT (1, 2, 3, ...)

3.2 Cle etrangere (FOREIGN KEY)

Une cle etrangere est une **reference** a la cle primaire d'une autre table. Elle garantit l'integrite referentielle.

```
CREATE TABLE filieres (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  code        VARCHAR(20) UNIQUE,
  name        VARCHAR(200) NOT NULL,
  department_id BIGINT NOT NULL,

  CONSTRAINT fk_filiere_department
    FOREIGN KEY (department_id)
      REFERENCES departments(id)
      ON DELETE RESTRICT -- empeche de supprimer un dept avec filieres
) ENGINE=InnoDB;
```

3.3 Les actions ON DELETE

Action	Effet
RESTRICT	Empeche la suppression si refs existent
CASCADE	Supprime tout en cascade (DANGEREUX)
SET NULL	Met la FK a NULL
NO ACTION	Comme RESTRICT

3.4 Exemple dans notre projet

departments (1)			filieres (N)		
id	code		id	code	dept_id
1	MI		1	SMI	1
2	PH		2	DI	1
3	CH		3	SMA	1
			4	SMP	2

SMI, DI, SMA pointent vers le departement MI (id=1)

Chapitre 4 - Normalisation (1NF, 2NF, 3NF)

4.1 Pourquoi normaliser ?

La **normalisation** est le processus d'organiser une BDD pour **eviter la redondance** et garantir la coherence. C'est la theorie qui distingue les amateurs des pros.

4.2 Exemple non-normalise (mauvais)

```
TABLE bad_students :
+---+-----+-----+-----+-----+
| id | name  | filiere | filiere_dept | filiere_chef |
+---+-----+-----+-----+-----+
| 1  | Karim | SMI     | Math-Info    | Pr. Alaoui   |
| 2  | Salma | SMI     | Math-Info    | Pr. Alaoui   |
| 3  | Hamza | SMI     | Math-Info    | Pr. Alaoui   |
+---+-----+-----+-----+-----+
```

Probleme : si on change le chef du dept, il faut modifier
TOUTES les lignes de Karim, Salma, Hamza...

4.3 Apres normalisation

```
TABLE departments :
+---+-----+-----+
| id | name    | chef      |
+---+-----+-----+
| 1  | Math-Info | Pr. Alaoui |
+---+-----+-----+
```

```
TABLE filieres :
+---+-----+-----+
| id | code | dept_id |
+---+-----+-----+
| 1  | SMI  | 1       |
+---+-----+-----+
```

```
TABLE students :
+---+-----+-----+
| id | name  | filiere_id |
+---+-----+-----+
| 1  | Karim | 1          |
| 2  | Salma | 1          |
| 3  | Hamza | 1          |
+---+-----+-----+
```

Maintenant : changer le chef = modifier UNE seule ligne.

4.4 Les 3 formes normales

1NF - Premiere forme normale. Chaque cellule contient UNE valeur (pas de listes). Chaque ligne est unique.

2NF - Deuxieme forme normale. 1NF + tous les attributs non-cle dependent de TOUTE la cle primaire.

3NF - Troisieme forme normale. 2NF + aucun attribut non-cle ne depend d'un autre attribut non-cle.

[✓] Notre projet respecte la 3NF. Cela rend les donnees coherentes, evite la duplication, et simplifie les mises a jour.

Chapitre 5 - Les 16 tables de fsbm_db

Voici la liste complete des tables et leur role :

- 1. departments.** Les 5 departements academiques. PK=id, UNIQUE=code (MI, PH, CH, BIO, GEO). Champs : name, description, head_name, head_email, color_hex pour l'UI.
- 2. filieres.** 25 filieres : 7 licences + 18 masters. PK=id, FK=department_id. Champs : code (SMI, IADS), type (LICENCE/MASTER), coordinator, capacity, careers.
- 3. modules.** 100+ modules pour chaque filiere. PK=id, FK=filiere_id. Champs : code (SMI-S1-M1), semester, credits, coefficient, hours_cours/td/tp.
- 4. professors.** 107 profs. PK=id, FK=department_id, UNIQUE=matricule, email. Champs : grade (PA/PH/PES), specialty, bureau, photo_url.
- 5. students.** 2970 etudiants. PK=id, FK=filiere_id, UNIQUE=cne, apogee, email. Champs : annee_etude, group_name, is_boursier, statut (ACTIF/SUSPENDU/DIPLOME).
- 6. module_teachers.** Relation N:N entre modules et professors. FK=module_id, FK=professor_id. Champs : role (TITULAIRE/ASSISTANT/TD/TP), annee_univ.
- 7. schedules.** Emploi du temps. FK=filiere_id, FK=module_id, FK=professor_id. Champs : day_of_week, start_time, end_time, salle, type_seance.
- 8. exams.** Calendrier examens. FK=module_id, FK=filiere_id. Champs : exam_date, salle, session (NORMALE_S1/S2/RATTRAPAGE).
- 9. grades.** 9000+ notes. FK=student_id, FK=module_id. Champs : note_cc, note_examen, note_finale, mention (PASSABLE/AB/BIEN/TB).
- 10. faq_categories.** 16 categories pour le chatbot. PK=id, UNIQUE=code. Champs : name, icon (emoji), color_hex.
- 11. faq_items.** Items FAQ avec FULLTEXT search. FK=category_id. Champs : intent_tag, question, answer, keywords.
- 12. conversations.** Historique chatbot. Champs : session_id, user_message, bot_response, intent_detected, confidence, response_time_ms.
- 13. feedbacks.** Notes 1-5 utilisateurs. FK=conversation_id. Champs : note, is_helpful, commentaire.
- 14. announcements.** Annonces officielles. FK=target_filiere (optionnel). Champs : title, content, type (INFO/URGENT/EXAMEN/EVENT/VACANCE), is_pinned.
- 15. events.** Evenements (JPO, hackathon...). Champs : title, event_type, start_date, location, registration_url.
- 16. clubs.** 8 clubs etudiants. Champs : name, category, president, social_links (JSON), members_count.

Chapitre 6 - Relations entre tables

6.1 Relation 1:N (un a plusieurs)

Un departement a PLUSIEURS filieres. Une filiere appartient a UN departement.

departments	filieres
+---+-----+	+---+-----+-----+
id code 1 -----N>	id code department_id
+---+-----+	+---+-----+-----+

6.2 Relation N:N (plusieurs a plusieurs)

Un module peut etre enseigne par PLUSIEURS profs. Un prof peut enseigner PLUSIEURS modules.
Solution : table intermediaire module_teachers.

modules	module_teachers	professors
+---+	+-----+-----+	+---+
id 1 ----N	mod_id prof_id role N---- 1	id
+---+	+-----+-----+	+---+

6.3 Toutes les relations de fsbm_db

departments (1) -----N	filieres -----N	modules
	v	v
	students	schedules
	v	v
	grades	exams
departments (1) -----N	professors -----N	module_teachers >----- modules
faq_categories (1) -----N	faq_items	
conversations (1) -----N	feedbacks	
filieres (1) -----0-N	announcements	

Chapitre 7 - Index et performance

7.1 C'est quoi un index ?

Un **index** est une **structure de donnees auxiliaire** qui permet de retrouver rapidement les lignes correspondant a une valeur. C'est comme la **table des matieres** d'un livre.

■ **ANALOGIE** : Sans index, MySQL lit toutes les lignes une par une jusqu'a trouver. C'est comme chercher 'apple' en lisant le dictionnaire de A a Z. Avec index, MySQL utilise un arbre B-tree (sorte d'index alphabetique) pour aller direct au bon endroit.

7.2 Quand creer un index ?

- Sur les colonnes utilisees dans des WHERE frequents
- Sur les FK (deja automatique en InnoDB)
- Sur les colonnes utilisees dans des ORDER BY
- PAS sur les colonnes peu selectives (ex: boolean)
- PAS sur des tables avec beaucoup d'INSERT (les index ralentissent l'ecriture)

7.3 Index dans notre projet

```
CREATE TABLE students (
  id          BIGINT PRIMARY KEY,
  cne         VARCHAR(20) UNIQUE,
  first_name  VARCHAR(80),
  last_name   VARCHAR(80),
  email       VARCHAR(150) UNIQUE,
  filiere_id  BIGINT,
  annee_etude TINYINT,

  -- Index explicites :
  INDEX idx_student_filiere (filiere_id),
  INDEX idx_student_year (annee_etude),
  INDEX idx_student_name (last_name, first_name), -- index compose
  INDEX idx_student_group (group_name)
);
```

7.4 FULLTEXT pour recherche texte

```
CREATE TABLE faq_items (
  ...,
  FULLTEXT idx_faq_search (question, answer, keywords)
);

-- Utilisation :
SELECT * FROM faq_items
WHERE MATCH(question, answer) AGAINST('inscription master');
```

Chapitre 8 - C'est quoi un ORM ?

8.1 Le probleme du SQL brut

```
# Approche raw SQL (pymysql)
conn = pymysql.connect(host='localhost', user='root', password='...')
cursor = conn.cursor()
cursor.execute("SELECT * FROM filieres WHERE type = %s", ('MASTER',))
rows = cursor.fetchall()

# Problemes :
# - Strings SQL fragiles (risque d'injection)
# - rows est juste une liste de tuples (pas tres lisible)
# - Pas de type checking
# - Difficile a maintenir
```

8.2 La solution ORM

ORM = Object-Relational Mapping. Au lieu d'ecrire du SQL, on manipule des **objets Python**. L'ORM traduit en SQL automatiquement.

```
# Approche ORM (SQLAlchemy)
result = await db.execute(
    select(Filiere).where(Filiere.type == 'MASTER')
)
masters = result.scalars().all()

for m in masters:
    print(m.code, m.name) # acces objet, autocomplete IDE
```

8.3 Avantages de l'ORM

- + **Securite** : protection auto contre les injections SQL
- + **Type safety** : autocompletion IDE, type checking
- + **Portabilite** : si tu changes de SGBD (MySQL -> PostgreSQL), peu de code a changer
- + **Productivite** : moins de code a ecrire
- + **Migrations** : Alembic genere les CREATE TABLE auto a partir des modeles

8.4 Inconvenients

- Performance : ORM est 10-30% plus lent que SQL natif
- Complexite : pour requetes complexes, l'ORM peut etre verbeux
- Courbe d'apprentissage : apprendre les concepts ORM

Chapitre 9 - SQLAlchemy 2.0

9.1 SQLAlchemy en bref

SQLAlchemy est l'ORM Python **standard**. Version 2.0 (2023) apporte une syntaxe moderne avec `Mapped[T]` et un support `async` natif.

9.2 Definition d'un modele

```
from sqlalchemy import String, BigInteger, ForeignKey
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Department(Base):
    __tablename__ = 'departments'

    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    code: Mapped[str] = mapped_column(String(20), unique=True)
    name: Mapped[str] = mapped_column(String(150))

    # Relation 1:N
    filieres: Mapped[list['Filiere']] = relationship(
        back_populates='department'
    )

class Filiere(Base):
    __tablename__ = 'filieres'

    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    code: Mapped[str] = mapped_column(String(20), unique=True)
    department_id: Mapped[int] = mapped_column(ForeignKey('departments.id'))

    # Relation inverse
    department: Mapped['Department'] = relationship(
        back_populates='filieres'
    )
```

9.3 Avantages de la syntaxe `Mapped[T]`

- Type hints natifs Python (autocompletion)
- Mypy/Pyright peuvent verifier les types
- Plus lisible que la syntaxe `Column()` de SQLAlchemy 1.x
- Optionnel automatique : `Mapped[Optional[str]] = NULL` OK

Chapitre 10 - SQLAlchemy async (aiomysql)

10.1 Pourquoi async ?

Sans async, chaque requete DB bloque le serveur pendant 1-100ms. Avec async, le serveur peut traiter d'autres requetes pendant l'attente.

10.2 Setup du moteur async

```
from sqlalchemy.ext.asyncio import (
    create_async_engine, async_sessionmaker, AsyncSession
)

DATABASE_URL = (
    'mysql+aiomysql://root:password@localhost:3306/fsbm_db'
)

engine = create_async_engine(
    DATABASE_URL,
    echo=False,          # True pour voir le SQL genere
    pool_pre_ping=True,  # verifie la connexion avant usage
    pool_recycle=3600,   # recycle apres 1h
)

SessionLocal = async_sessionmaker(
    engine,
    expire_on_commit=False,
    class_=AsyncSession,
)
```

10.3 Dependency pour les routes

```
async def get_db() -> AsyncSession:
    async with SessionLocal() as session:
        yield session
    # Apres yield : la session se ferme automatiquement

@router.get('/api/filieres')
async def list_filieres(db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(Filiere))
    return result.scalars().all()
```

10.4 Pool de connexions

SQLAlchemy maintient un **pool** de connexions reutilisables. Au lieu d'ouvrir/fermer une connexion par requete (couteux), on emprunte une connexion existante du pool.

 Par default : 5 connexions max + 10 overflow. Pour 1000 utilisateurs concurrents, augmenter pool_size=20, max_overflow=30.

Chapitre 11 - CRUD avec SQLAlchemy

11.1 CREATE (Insert)

```
new_dept = Department(code='INFO', name='Informatique nouvelle')
db.add(new_dept)
await db.commit()
await db.refresh(new_dept)
print(new_dept.id) # ID auto-generé
```

11.2 READ (Select)

```
# Tout
result = await db.execute(select(Filiere))
all_filiere = result.scalars().all()

# Par ID
fil = await db.get(Filiere, 1)

# Avec WHERE
stmt = select(Filiere).where(Filiere.type == 'MASTER')
result = await db.execute(stmt)
masters = result.scalars().all()

# Avec JOIN
stmt = (
    select(Filiere, Department.name.label('dept_name'))
    .join(Department, Department.id == Filiere.department_id)
)
result = await db.execute(stmt)
for fil, dept_name in result.all():
    print(fil.code, dept_name)

# Count
total = await db.scalar(select(func.count(Filiere.id)))

# Group by
stmt = (
    select(Filiere.type, func.count(Filiere.id))
    .group_by(Filiere.type)
)
result = await db.execute(stmt)
for type_, count in result.all():
    print(type_, count)
```

11.3 UPDATE

```
# Methode 1 : update objet + commit
fil = await db.get(Filiere, 1)
fil.capacity = 200
await db.commit()

# Methode 2 : UPDATE en masse
from sqlalchemy import update
stmt = (
    update(Filiere)
```

```
        .where(Filiere.type == 'MASTER')
        .values(is_active=True)
    )
    await db.execute(stmt)
    await db.commit()
```

11.4 DELETE

```
# Methode 1 : delete objet
fil = await db.get(Filiere, 99)
if fil:
    await db.delete(fil)
    await db.commit()

# Methode 2 : DELETE en masse
from sqlalchemy import delete
stmt = delete(FaqItem).where(FaqItem.consultations == 0)
await db.execute(stmt)
await db.commit()
```


Chapitre 12 - Pagination

12.1 Pourquoi paginer ?

Sans pagination, GET /api/students renvoie 2970 etudiants d'un coup. C'est :

- Lent (transfert reseau lourd)
- Memory-intensive cote client
- Inutile (l'utilisateur regarde 20 etudiants a la fois)

12.2 Pagination avec offset/limit

```
@router.get('/api/students', response_model=PaginatedResponse)
async def list_students(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: AsyncSession = Depends(get_db),
):
    # Count total
    total = await db.scalar(select(func.count(Student.id))) or 0

    # Paginer
    stmt = (
        select(Student)
        .order_by(Student.last_name, Student.first_name)
        .offset((page - 1) * page_size)
        .limit(page_size)
    )
    result = await db.execute(stmt)
    items = result.scalars().all()

    return PaginatedResponse(
        items=items,
        total=total,
        page=page,
        page_size=page_size,
        total_pages=(total + page_size - 1) // page_size,
    )
```

12.3 Schema PaginatedResponse

```
from pydantic import BaseModel
from typing import Generic, TypeVar

T = TypeVar('T')

class PaginatedResponse(BaseModel, Generic[T]):
    items: list[T]
    total: int
    page: int = 1
    page_size: int = 20
    total_pages: int = 1

# Usage avec type fort :
```

```
@router.get('/api/students',  
            response_model=PaginatedResponse[StudentOut])  
async def list_students(...): ...
```

Chapitre 13 - Conclusion

13.1 Recap

- + BDD relationnelle = donnees structurees en tables liees par FK
- + SQL = langage standard pour parler aux BDD
- + Cles primaires (uniques) et etrangeres (references)
- + Normalisation 3NF = pas de redondance
- + Index = recherche rapide (B-tree, FULLTEXT)
- + ORM = manipuler des objets Python au lieu de SQL
- + SQLAlchemy 2.0 = ORM moderne async avec Mapped[T]
- + Async + pool de connexions = perfs max
- + CRUD = Create/Read/Update/Delete
- + Pagination = ne pas tout renvoyer d'un coup

13.2 Pour aller plus loin

- PDF 05 - pour MongoDB et les differences SQL vs NoSQL
- PDF 03 - pour comprendre comment FastAPI s'integre avec SQLAlchemy
- PDF 09 - pour le flux complet incluant une requete DB